

Instalación de un Contenedor Web de Libre Distribución

Node.js + Express (TypeScript) en
Linux, macOS y Windows

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Lunes 10 de agosto de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Instalar Node.js, npm y TypeScript en tu sistema operativo, de forma correcta y verificable.
2. Crear un proyecto Express en TypeScript desde cero, entendiendo cada archivo de configuración.
3. Configurar el contenedor como **servicio del sistema** (igual que cualquier software de servidor real).



Conexión con la Clase 6

Recordatorio: en la Clase 6 desplegamos una aplicación TypeScript dentro de IIS usando iisnode – nuestro contenedor **propietario**.

Hoy: repetimos el ejercicio de instalación, ahora con nuestro contenedor de **libre distribución**: Node.js + Express.

En IIS tuvimos que instalar un módulo adicional (iisnode) para que aceptara código Node – aquí Node.js no es un invitado: corre nativo en Linux, macOS o Windows, sin intermediarios.



¿Por qué Node.js + Express?

En este módulo usamos **Node.js + Express** como nuestro contenedor de libre distribución porque:

1. Es **TypeScript de principio a fin** – mismo lenguaje en las 3 unidades del curso, sin cambiar de paradigma a mitad de camino.
2. Es, hoy, el contenedor de libre distribución **más usado en la industria** para construir APIs y servicios web (Netflix, Uber, LinkedIn lo usan en producción).
3. Cumple **exactamente el mismo rol** que cualquier otro contenedor: recibe peticiones HTTP, decide qué código las atiende, ejecuta esa lógica. La teoría sobre "contenedores y servlets" aplica igual – solo cambia el lenguaje de implementación.



Requisitos

- Acceso de administrador (sudo en Linux/macOS, PowerShell como Administrador en Windows).
- Conexión a internet.
- **Cualquier** distribución de Linux, macOS o Windows – esta clase no exige un sistema específico.

Cada paso de hoy trae su comando para los tres sistemas. Siguen la columna que corresponda a su equipo – el resultado final (el contenedor corriendo como servicio) es idéntico en los tres.



Paso 1: instalar Node.js y npm

Linux

```
1 sudo apt install  
2 -y nodejs npm
```

(o dnf/pacman según tu distro)

macOS

```
1 brew install node
```

(con Homebrew instalado)

Windows

```
1 winget install  
2 OpenJS.NodeJS
```

(PowerShell)

Verificar en cualquier sistema:

```
1 node -v  
2 npm -v
```

¿Qué es npm?

El "administrador de paquetes" de Node – una tienda de aplicaciones, pero para librerías de código gratuitas que otros programadores ya escribieron.



Paso 2: instalar TypeScript globalmente

Este paso es **idéntico en los tres sistemas** – una vez Node.js/npm están instalados, TypeScript se instala igual en todos:

```
1 npm install -g typescript ts-node  
2 tsc -v
```

"Global" (-g) significa que el comando tsc queda disponible en cualquier carpeta del sistema, no solo dentro de un proyecto.



Paso 3: usuario de servicio dedicado

Buena práctica de seguridad: **nunca** correr una aplicación con tu usuario administrador.

Linux

```
1 sudo useradd -m -U
2 -s /usr/sbin/nologin
3 appweb
```

macOS

```
1 sudo sysadminctl
2 -addUser appweb
3 -shell /usr/bin/false
```

Windows (PowerShell)

```
1 New-LocalUser -Name "appweb" -
   NoPassword
```

¿Por qué un usuario aparte?

Si la aplicación tuviera una falla de seguridad, que corra como un usuario **sin privilegios** limita el daño – no podría, por ejemplo, borrar archivos de todo el sistema.



Paso 4: crear el proyecto Express

La carpeta cambia de ubicación según el sistema, pero los comandos de npm son **idénticos** en los tres:

```
1 npm init -y
2 npm install express
3 npm install --save-dev typescript @types/express @types/node
4 npx tsc --init
```

Linux/macOS: /opt/miapp
(equivalente al C:\inetpub de IIS)

Windows: C:\miapp



Paso 4 (cont.): servidor.ts

```
1 import express, { type Request, type Response } from "express";
2
3 const app = express();
4 const PUERTO = 3000;
5
6 app.get("/", (req: Request, res: Response) => {
7   res.send("<h1>Contenedor de libre distribucion activo " +
8     "(Node.js + Express + TypeScript)</h1>");
9 });
10
11 app.listen(PUERTO, () => {
12   console.log(`Escuchando en http://localhost:${PUERTO}`);
13 });
```

Este código es idéntico sin importar el sistema operativo – `express()` reemplaza nuestro `http.createServer` manual de la Clase 2.



Compilar y probar manualmente

Antes de instalarlo como servicio, comprobar que funciona a mano – mismo comando en los 3 sistemas:

```
1 tsc servidor.ts  
2 node servidor.js
```

Visitar <http://localhost:3000> – confirmar que aparece el mensaje.

*Mismo patrón de la Clase 6: TypeScript nunca corre directo, siempre se compila a .js primero. Ctrl+C detiene esta prueba manual – el siguiente paso lo deja corriendo **solo**, sin terminal abierta.*



¿Qué es un "servicio del sistema"?

En la Clase 5 vimos que Windows mantiene IIS corriendo todo el tiempo mediante un **servicio** (W3SVC). Cada sistema operativo tiene su propio mecanismo para esto:

Linux
systemd

macOS
launchd

Windows
Servicios de Windows (vía
NSSM)

Por qué importa

Si solo ejecutamos `node servidor.js` en la terminal, el contenedor **muere** en cuanto cerramos esa terminal o reiniciamos el equipo. Un servicio del sistema lo mantiene corriendo **solo**, lo reinicia si falla, y lo levanta automáticamente al encender.



Paso 5: crear el servicio – Linux (systemd)

Crear /etc/systemd/system/miapp.service:

```
1 [Unit]
2 Description=Contenedor Node.js/Express (TypeScript) - MiApp
3 After=network.target
4
5 [Service]
6 Type=simple
7 User=appweb
8 WorkingDirectory=/opt/miapp
9 ExecStart=/usr/bin/node /opt/miapp/servidor.js
10 Restart=on-failure
11
12 [Install]
13 WantedBy=multi-user.target
```



Paso 5: crear el servicio – macOS (launchd)

Crear /Library/LaunchDaemons/com.miapp.plist:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <plist version="1.0">
3 <dict>
4   <key>Label</key><string>com.miapp.servidor</string>
5   <key>ProgramArguments</key>
6   <array>
7     <string>/usr/local/bin/node</string>
8     <string>/opt/miapp/servidor.js</string>
9   </array>
10  <key>UserName</key><string>appweb</string>
11  <key>RunAtLoad</key><true/>
12  <key>KeepAlive</key><true/>
13 </dict>
14 </plist>
```

RunAtLoad + KeepAlive hacen el mismo trabajo que WantedBy + Restart=on-failure en systemd.



Paso 5: crear el servicio – Windows (NSSM)

Windows no incluye una forma simple de convertir un programa en servicio – se usa **NSSM** (*Non-Sucking Service Manager*), herramienta gratuita y estándar de la industria para esto:

```
1 nssm install miapp "C:\Program Files\nodejs\node.exe" "C:\miapp\servidor.js"  
2 nssm set miapp AppDirectory "C:\miapp"  
3 nssm set miapp ObjectName ".\appweb"
```

nssm install abre un asistente equivalente al *web.config* de IIS: define qué ejecutar, desde dónde, y con qué usuario.



Paso 6: habilitar y arrancar el servicio

Linux

```
1 sudo systemctl
2   daemon-reload
3 sudo systemctl
4   enable --now miapp
5 sudo systemctl
6   status miapp
```

macOS

```
1 sudo launchctl load -w /Library/
   LaunchDaemons/com.miapp.plist
2 sudo launchctl list | grep miapp
```

Windows

```
1 nssm start miapp
2 Get-Service miapp
```

Resultado esperado en los tres: el servicio aparece **activo/corriendo**, y `http://localhost:3000` responde sin tener ninguna terminal abierta.



El camino completo, de un vistazo

1. Instalar Node.js, npm y TypeScript (comando según tu SO)



2. Crear usuario de servicio appweb (sin privilegios)



3. `npm init + express + escribir servidor.ts`



4. Compilar y probar a mano: `tsc + node servidor.js`



5. Crear el servicio (systemd / launchd / NSSM)



6. Habilitar y arrancar el servicio



7. Corriendo solo, sin terminal abierta





IIS (Clase 6) vs. Node/Express (hoy): mismo patrón

Concepto	IIS (propietario)	Node/Express (libre)
Sistema operativo	Windows Server	Linux / macOS / Windows
Mecanismo de servicio	W3SVC (ya incluido)	systemd / launchd / NSSM
Aislamiento de proceso	Application Pool	Usuario dedicado appweb
Compilar TS → JS	tsc app.ts	tsc servidor.ts

Mismo problema (mantener un contenedor corriendo de forma confiable), distintas soluciones según el sistema – exactamente la idea de la Clase 4.



Ejercicio individual

1. Completar la instalación siguiendo todos los pasos, en el sistema operativo que ustedes usen.
2. Capturar pantalla de:
 - `node -v` y `tsc -v`.
 - El servicio mostrando estado activo/corriendo.
 - La respuesta en el navegador en `http://localhost:3000`.
3. Detener el servicio, confirmar que el sitio deja de responder, y volver a iniciarlo. Anotar qué observaron.





Pista para el ejercicio

Pista

Al detener el servicio, el navegador debería mostrar un error de conexión (no "404", sino "no se puede conectar") – esa es la diferencia entre "el contenedor responde pero no encuentra la ruta" y "no hay nadie escuchando en ese puerto".

Si después de arrancarlo de nuevo no responde, revisen el estado del servicio (`systemctl status / launchctl list / Get-Service`) – casi siempre el error ahí mismo dice qué falló.



Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?





Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?

Principio de menor privilegio: limita el daño si el proceso es comprometido.





Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?

Principio de menor privilegio: limita el daño si el proceso es comprometido.

2. ¿Qué hace tsc?





Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?

Principio de menor privilegio: limita el daño si el proceso es comprometido.

2. ¿Qué hace tsc?

Compila TypeScript a JavaScript, el lenguaje que Node.js ejecuta realmente.





Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?

Principio de menor privilegio: limita el daño si el proceso es comprometido.

2. ¿Qué hace tsc?

Compila TypeScript a JavaScript, el lenguaje que Node.js ejecuta realmente.

3. ¿Qué ventaja da usar un servicio del sistema en lugar de simplemente ejecutar `node servidor.js` en la terminal?



Verificación de aprendizaje

1. ¿Por qué se crea un usuario dedicado (appweb) en lugar de correr con el usuario administrador?

Principio de menor privilegio: limita el daño si el proceso es comprometido.

2. ¿Qué hace tsc?

Compila TypeScript a JavaScript, el lenguaje que Node.js ejecuta realmente.

3. ¿Qué ventaja da usar un servicio del sistema en lugar de simplemente ejecutar `node servidor.js` en la terminal?

El servicio arranca automáticamente al reiniciar el equipo, y se reinicia solo si falla.



Resumen de la clase

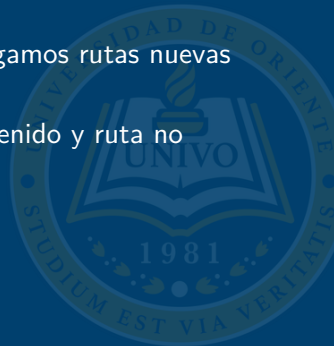
Paso	Comando clave
Instalar Node/npm	apt / brew / winget según tu SO
Instalar TypeScript	npm install -g typescript ts-node
Usuario de servicio	useradd / sysadminctl / New-LocalUser
Crear proyecto	npm init -y + npm install express
Servicio del sistema	systemd / launchd / NSSM



Preparación para la Clase 8

Tema: Construcción y despliegue de rutas dinámicas (el equivalente TypeScript a un "servlet") en nuestro contenedor Express.

- Dejar miapp corriendo como servicio – la próxima clase agregamos rutas nuevas sobre esta misma base.
- Repasar el ejercicio individual: la diferencia entre servicio detenido y ruta no encontrada se va a usar de nuevo.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

